

Lifting DecPOMDPs for Nanoscale Systems — A Work in Progress

Tanya Braun¹, Stefan Fischer², Florian Lau², Ralf Möller¹

¹ Institute of Information Systems, University of L  ijbeck, L  ijbeck, Germany

² Institute of Telematics, University of L  ijbeck, L  ijbeck, Germany
{tanya.braun, stefan.fischer, florian.lau, moeller}@uni-luebeck.de

Abstract

DNA-based nanonetworks have a wide range of promising use cases, especially in the field of medicine. With a large set of agents, a partially observable stochastic environment, and noisy observations, such nanoscale systems can be modelled as a decentralised, partially observable, Markov decision process (DecPOMDP). As the agent set is a dominating factor, this paper presents (i) lifted DecPOMDPs, partitioning the agent set into sets of indistinguishable agents, reducing the worst-case space required, and (ii) a nanoscale medical system as an application.

Particularly in times of medical crisis, precise and efficient diagnostic tools are invaluable. The recent development of DNA-based nanonetworks shows promising results as a fast and robust diagnostics tool (Lau, Wendt, and Fischer 2021) and a treatment method for diseases using nanobots in the human body (Lau et al. 2019). Such tasks are realised by a swarm of thousands of nanodevices collaborating to compute a diagnosis or treat a disease. However, a network’s environment is extremely heterogenous. Thus, the nanodevices have only local information about the system, which they are not able to consolidate into a global state, with their communication and computation capabilities highly limited and local information subject to noise. Under these conditions, a designer of a nanonetwork has to set up nanodevices functioning according to a certain plan or policy. In addition, the designer needs to assess, e.g., the network’s robustness regarding a diagnosis or the success rate of a joint action of medicine delivery.

Therefore, this paper formalises the setting as an offline decision making problem, specifically, a decentralised partially observable Markov decision process (DecPOMDP). The formalisation fits well as a network contains a set of collaborating nanodevices, i.e., *agents*, with limited capabilities in a *partially observable* environment whose transition can be approximated with a *stochastic* process. Computation time in DecPOMDPs depends exponentially on the number of agents in the worst case, which is problematic as nanonetworks can have hundreds of thousands of agents.

However, only a limited number of types of nanodevices exists, thus, partitioning the set of agents into subsets of agents whose behaviour is *indistinguishable*. The notion of *lifting* refers to the idea of efficiently handling sets of indistinguishable objects using representatives (Poole 2003), reducing the theoretical dependency from exponential to polynomial w.r.t. these set sizes. Here, we apply lifting to the agents of a DecPOMDP. Specifically, the contributions of this paper are twofold: (i) *lifted DecPOMDPs* with a theoretical analysis in terms of space requirements and (ii) a *nanoscale medical system* as an application, forming a first step towards our long-term goal of a full formalisation combined with a lifted solution approach that allows for quantifying a joint action’s success rate.

Lifting has been successfully used for probabilistic inference (see, e.g., Van den Broeck et al. 2011; Ahmadi et al. 2013; Braun and M  ller 2018; Holtzen, Millstein, and Van den Broeck 2019) as well as online decision making (Nath and Domingos 2009; Apsel and Brafman 2011; Gehrke et al. 2019; Gehrke, Braun, and M  ller 2019), which uses decision and utility nodes in a lifted probabilistic graphical model (PGM). Lifting can even bring tractability in terms of the set sizes of indistinguishable objects (Niepert and Van den Broeck 2014). In offline decision making, lifting has been used in calculations for relational descriptions of (PO)MDPs: First-order MDPs (FOMDPs, Boutilier, Reiter, and Price 2001; Sanner and Boutilier 2009) have a representation based on the situation calculus (McCarthy 1963). Sanner and Kersting (2010) use lifting for pruning indistinguishable policies in their solution approach for a partially observable FOMDP. Factorised FOMDP assume a factorisation of its representation, blurring the line towards online decision making with the factored representation (Sanner and Boutilier 2007). First-order open-universe POMDPs follow the open-universe assumption for the first-order description of an environment using Bayesian logic as a basis (Srivastava et al. 2014). Another first-order representation is the language of independent choice logic that also allows for a set of agents (Poole 1997). To the best of our knowledge, we are the first to consider lifting the agent set, which finds its application in nanoscale systems where large

groups of indistinguishable nanodevices act jointly in an environment. We leave the environment encoded by a non-lifted joint distribution in this paper since the application at hand does not lend itself to a highly structured model. Nonetheless, dealing with structure in the environment, relational or factorised, presents an exciting avenue for future work to extend its applicability.

The remainder of this paper is structured as follows: We start with an introduction to nanoscale medical systems and recap offline decision-making from MDPs to DecPOMDPs. Then, we present lifted DecPOMDPs, followed by a discussion of a lifted DecPOMDP modelling a nanosystem and next steps towards solving such a problem. We end with a conclusion.

Nanoscale Medical Systems

DNA-based nanonetworks have been proposed as an alternative to polymerase chain reaction, PCR for short, for detecting arbitrary diseases on the basis of DNA. In this scenario, a disease sample is mixed with a medical nanosystem that computes a programmed function depending on environmental parameters to decide if a disease is present. This section explains the basic mechanisms and ideas behind this novel technology and sets up how DecPOMDPs can be used as a model for it.

In general, a computational process at nanoscale is subject to resource constraints, and collaboration between nanodevices might be necessary to achieve a goal (Akyildiz, Brunetti, and BIAzquez 2008). In 1982, Seeman first proposed DNA as a construction material for nanoscale systems. Based on this idea, Rothemund (2006) developed the DNA-origami method, which allows for creating almost arbitrary shapes using DNA at nanoscale. In 2009, Andersen et al. created a box with a controllable lid using the DNA-origami method. These boxes serve as a basis for medical nanosystem technology. They can be filled with either medication or DNA-tiles that serve as an input for a computation (Winfrey et al. 1998). Certain DNA-tile systems form a Turing-complete computational model and are much more capable than most widely used medical diagnostic tools (Lau et al. 2019). For a detailed introduction, we refer to Winfree et al. (1998).

Figures 1a and 1b show an example system of DNA-tiles that computes a 4-bit AND operation. The blocks in Fig. 1a represent DNA-tiles. They are modelled as non-rotatable blocks with colour-coded *glues* at possibly all sides. The number of black boxes represents the *strength* of glue and a label next to it a condition. Tiles with the same number of boxes and identical labels can form a binding. That binding is subject to an environmental parameter called *temperature* τ . The temperature τ encodes the necessary number of fitting glues over all neighbours to form a stable binding. In the presented example, the temperature is 2, and all tiles need at least two neighbours to stably bind together.

An assembly process begins with a *seed-tile* σ , shown in Fig. 1b at the right. Due to the temperature requirement of 2, three tiles can bind to the seed-tile σ : tile T,

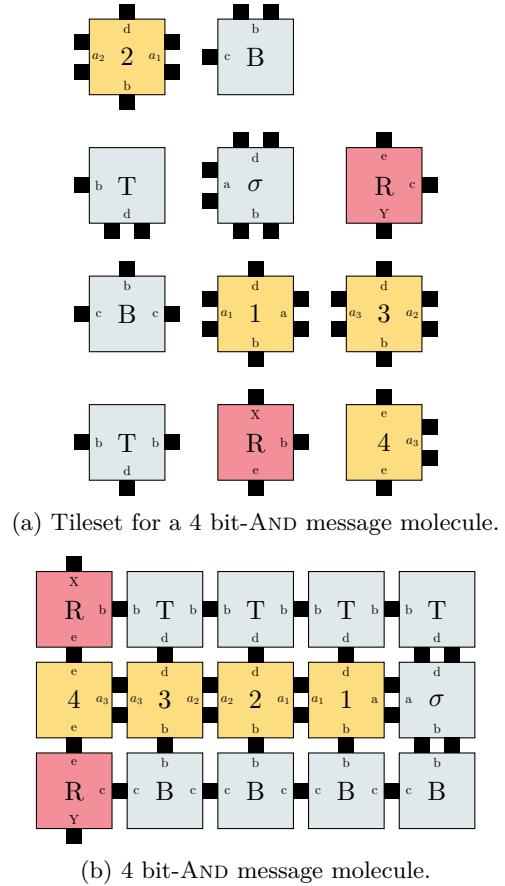


Figure 1: Tileset and finished product of an assembly process of DNA-tiles.

B, or 1. The assembly process continues until the entire DNA molecule in Fig. 1b is formed. The *receptors* R can only bind with the molecule if tiles 1, 2, 3, and 4 are present. If those tiles are only conditionally present, they can represent the input values of a computation, in this case of a 4-bit AND. The molecule can only fully assemble if all four tiles are present. The other tiles are assumed to be present at all times in the medium.

Figure 2 shows the same DNA molecule assembly process incorporated into a network structure. In the first phase, a possibly large number of predefined markers are detected by a very large number of indistinguishable nanosensors of four different types. Upon detection, the lids of the nanosensors open, and they release their tiles 1–4 into the medium. If all four tiles are present in high enough numbers, message molecules can fully assemble and later be detected by a number of indistinguishable nanobots. Those react by predefined programming, e.g., releasing a fluorescence marker, medication, or additional tiles. Many operations are possible depending on the desired response by the nanonetwork.

The entire assembly process is guided by diffusion and Brownian motion. It can only be controlled by the types of supplied tiles, their concentration, and the en-

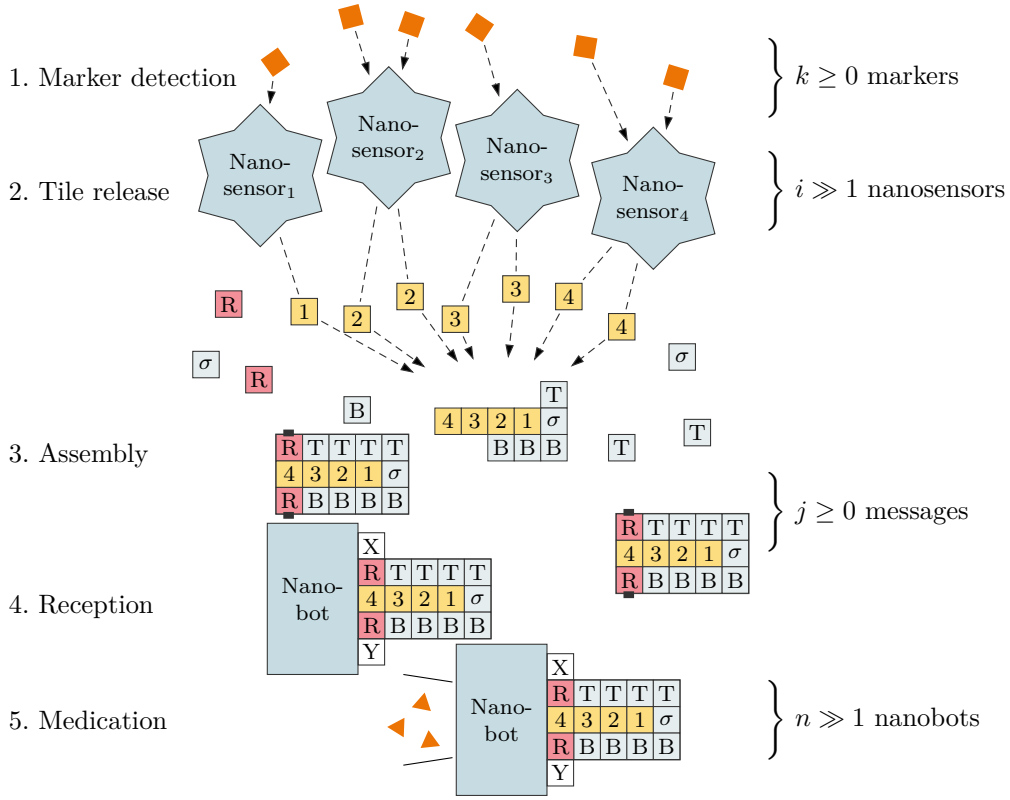


Figure 2: Nanonetwork, which detects markers and assembles a 4 bit-AND to achieve a distributed consensus.

environmental temperature. Due to the stochastic nature, it is never fully clear where or when a message molecule forms and if it is erroneous. Further, the nanodevices themselves have no global and very limited local information that is also noisy about their environment.

Now that the basic functionality of DNA-based nanonetworks is clear, we can model part of them as DecPOMDPs. The goal of this model is to coordinate the actions of the large number of indistinguishable devices in the nanonetwork to decide a predefined problem. Since the nanobots are extremely resource-constrained, they only have partial information about the global state of the system. In addition to that, communication in nanonetworks is expensive and should be reduced to a minimum. One of the potential benefits of the proposed model is to get an estimate of the success rate of such a system.

From Single Agent to Multiple Agents

This section highlights the genesis of offline decision making from single-agent-fully-observable to multi-agent-partially-observable-joint-reward, which forms the basis for modelling nanoscale systems. The underlying principle is that of *maximum expected utility* (MEU) in which a utility function represents preferences over states. We look through a PGM lens when setting up definitions, which are based on Russell and

Norvig (2020), using random variables, R , which can take discrete values, referred to as range, $ran(R) = \{r_1, \dots, r_m\}$. If R is set to a value of its range $r \in ran(R)$, also called an *event*, we denote it as $R = r$ or r for short if R is clear from its context. Random variables that have actions as ranges are called *decision random variables*. For solving an MDP or one of its extensions, we focus on *value iteration* as an example solution approach. For a more detailed look into different solution techniques, please refer to Oliehoek and Amato (2016) as a starting point with a focus on DecPOMDPs.

MDP An MDP is a sequential decision problem in a fully observable environment with a Markov-1 transition model and additive rewards, which implies that an agent's preferences are independent w.r.t. time.

Definition 1. An MDP is a tuple (S, A, T, R) , with

- S a random variable describing an agent's environment (the state space is the range of S),
- A_s a decision random variable with a set of actions for each state $s \in ran(S)$ as its range,
- $T(S', S, A_S) = P(S' | S, A_S)$ a transition model (if $a \notin ran(A_s)$ for a given s , then $P(s' | s, a) = 0$), and
- $R(S)$ a reward function.

The solution to an MDP is a function $\pi : ran(S) \mapsto ran(A_S)$ called a policy.

The reward may also depend on the action applied as well as the resulting state, which does not change the problem itself in a major way. Additionally, one can specify a *discount* factor $\gamma \in]0, 1]$. A discount factor close to 0 signifies that future rewards are irrelevant. The factor corresponds to an interest rate of $1 - \gamma/\gamma$. The utility of a state $s \in \text{ran}(S)$ can be described using the Bellman equation:

$$U(s) = R(s) + \gamma \max_{a \in \text{ran}(A_s)} \sum_{s' \in \text{ran}(S)} P(s' | s, a) U(s'). \quad (1)$$

The inner sum adds up *expected utilities* of each state s' , i.e., the utility of s' multiplied with the probability of reaching s' from s applying an action a . The optimal policy π^* is then given by:

$$\pi^*(s) = \arg \max_{a \in \text{ran}(A_s)} \sum_{s' \in \text{ran}(S)} P(s' | s, a) U(s'),$$

i.e., the agent chooses the a that yields the maximum expected utility. A standard approach to solving an MDP is called *value iteration*. It uses the bellman equation (Eq. (1)) in an update version:

$$U_{t+1}(s) \leftarrow R(s) + \gamma \max_{a \in \text{ran}(A_s)} \sum_{s' \in \text{ran}(S)} P(s' | s, a) U_t(s'),$$

updating the utilities until a stopping criterion is fulfilled, usually based on γ and an allowed error ϵ , e.g., $\max_s |U_{t+1}(s) - U_t(s)| < \epsilon(1 - \gamma)/\gamma$.

POMDP If an agent cannot reliably observe or is not able to fully determine its state, the state is considered latent or partially observable, which leads to POMDPs.

Definition 2. A POMDP is a tuple (S, A, T, R, O, Ω) with

- S, A, T, R the components of an MDP,
- O a random variable with a set of possible observations as a range, and
- $\Omega(O, S) = P(O | S)$ a sensor model.

A belief MDP is a tuple (B, A, T, R, O, Ω) , i.e., a POMDP where the latent S is replaced by random variable B with an infinite set of belief states as range, with a belief state $b \in \text{ran}(B)$ referring to a probability distribution over S . The state variables S, S' in T, R , and Ω are replaced by belief state variables B, B' .

The sensor model, analogously to the reward function, may also depend on an action as well as the outcome state. To solve a POMDP, a corresponding belief MDP is solved, where optimal actions in an optimal policy depend on belief states (instead of actual states). The solution to a belief MDP is called a conditional plan that combines **if...then...else** clauses, with the **if** conditions depending on observations and the bodies of the **if** and **else** parts containing actions. Each conditional plan p has a depth d and is recursively constructed using plans of depth $d - 1$:

$$U_p(s) = R(s) + \gamma \sum_{s' \in \text{ran}(S)} P(s' | s, a) \sum_{o \in \text{ran}(O)} P(o | s') U_{p.o}(s') \quad (2)$$

where a is the initial action in p and $p.o$ refers to the subplan of depth $d - 1$ for percept o that follows after a . Equation (2) yields a value iteration algorithm. All plans form hyperplanes in the belief space. Those plans that have the highest values for some area of the space are dominating plans, whereas dominated plans have lower values over every possible belief state. An integral part of value iteration is to eliminate dominated plans, which means for the recursive construction that fewer conditional plans need to be considered.

DecPOMDP In contrast to the single-agent setting so far, a DecPOMDP models a set of agents working jointly towards a common goal, a scenario relevant for nanoscale medical systems.

Definition 3. A DecPOMDP is a tuple $(\mathbf{I}, S, \{A_i\}_{i \in \mathbf{I}}, T, R, \{O_i\}_{i \in \mathbf{I}}, \Omega)$, with

- $\mathbf{I}$ a set of N agents,
- S a random variable with a set of states as range,
- A_i a decision random variable with a set of local actions as range for each agent $i \in \mathbf{I}$, with $\mathbf{A} = \times_{i \in \mathbf{I}} \text{ran}(A_i)$ the set of joint actions,
- $T(S', S, \mathbf{A}) = P(S' | S, \mathbf{A})$ a transition model,
- $R(S)$ a reward function,
- O_i a random variable with a set of local observations as range for each agent $i \in \mathbf{I}$, with $\mathbf{O} = \times_{i \in \mathbf{I}} \text{ran}(O_i)$ the set of joint observations, and
- $\Omega(\mathbf{O}, S) = P(\mathbf{O} | S)$ a sensor model.

Each agent $i \in \mathbf{I}$ has a local policy π_i . The solution to a DecPOMDP is a joint policy $\pi = (\pi_i)_{i \in \mathbf{I}}$.

In a DecPOMDP, each agent has its own set of actions and possible observations¹ whereas the state and reward function are joint. The joint state is usually assumed to not be fully observable, even if combining all local observations. This is also the case for nanoscale systems. If the joint state and reward function can be split up into mostly independent subspaces per agent, the DecPOMDP decomposes into a set of POMDPs that can be solved individually. The joint reward function encodes that the set of agents receives a reward from the environment as a team in contrast to individual rewards for each agent. The generalisation of a DecPOMDP is a partially observable stochastic game in which each agent has its own reward function $R_i(S)$, which possibly conflicts with other agents' rewards.

A straight-forward way to find a solution to a DecPOMDP is to build conditional plans for each agent using value iteration and eliminate dominated plans considering all agents. This approach quickly runs into memory problems because of the explosion of agent numbers, possible states, actions, and observations. To formalise the space requirements in a DecPOMDP, we set up a simple lemma, which makes the combinatorial explosion by the agent numbers explicit.

¹ $A_i = A_j$, $i, j \in \mathbf{I}$, is possible but not mandatory. The same holds for the sets of local observations.

Lemma 1. *The worst-case memory required for specifying a DecPOMDP of Def. 3 is exponential in N .*

Proof. The transition model $T(S', S, \mathbf{A})$ and sensor model $\Omega(\mathbf{O}, S)$ of a DecPOMDP of Def. 3 have sizes S_T^{dec} and S_Ω^{dec} , respectively, that lie in

$$S_T^{dec} \in O(s \cdot s \cdot a^N) \quad \text{and} \quad S_\Omega^{dec} \in O(s \cdot o^N), \quad (3)$$

with $s = |\text{ran}(S)|$, $a = \max_{i \in \{1, \dots, N\}} |\text{ran}(A_i)|$, and $o = \max_{i \in \{1, \dots, N\}} |\text{ran}(O_i)|$, which is exponential in the number of agents N . $\square$

For cases, in which the agent set carries structure in the form of partitions of indistinguishable agents, we present lifted DecPOMDPs as a step towards making the problem tractable.

Lifting DecPOMDPs

Modeling a nanoscale system as a DecPOMDP yields a large set of agents $\mathbf{I}$ with partitions forming in which agents have the same setup regarding available actions and possible observations. To get a handle on the large $\mathbf{I}$, we apply lifting to $\mathbf{I}$, using the identical setup of partitions for a compact encoding. Before we present lifted DecPOMDPs, we need to consider under what conditions we can lift the agent set and what consequences this lifting has for the different components of a DecPOMDP. Therefore, we first define a liftable DecPOMDP, then discuss its consequences, yielding a definition of a lifted DecPOMDPs. We end with a look at worst-case space requirements.

Definition 4. *A liftable DecPOMDP is a DecPOMDP in which the set of agents $\mathbf{I}$ is partitioned into K sets $\mathcal{J}$, i.e., $\mathbf{I} = \bigcup_{k=1}^K \mathcal{J}_k$ and $\forall k, l \in \{1, \dots, K\} : \mathcal{J}_k \cap \mathcal{J}_l = \emptyset$, where in each partition $\mathcal{J}_k$, it holds that $\forall i, j \in \mathcal{J}_k$:*

$$\text{ran}(A_i) = \text{ran}(A_j) \wedge \text{ran}(O_i) = \text{ran}(O_j). \quad (4)$$

As a consequence of Eq. (4), it is sufficient to use K variables A_k and O_k that apply to all agents in a partition $\mathcal{J}_k$ instead of A_i and O_i for all $i \in \mathbf{I}$. One may even explicitly model this pattern by using $A_k(X_k)$ and $O_k(X_k)$ with X_k a logical variable that represents all agents in $\mathcal{J}_k$ (for the use of logical variables in random variables, see, e.g., the work by Taghipour (2013)).

With a partitioning of the agent set, the joint observation $\mathbf{o} = (o_1, \dots, o_N)$ consists of K *partition observations* $\mathbf{o}_k = (o_{k,1}, \dots, o_{k,|\mathcal{J}_k|})$. Due to the ranges being discrete and bounded, each $\mathbf{o}_k$ can be lifted as each $o \in \mathbf{o}_k$ can only be in $\text{ran}(O_k)$. Picking up the idea of histograms, which are used by the lifting tool of counting (see the work by Milch et al. (2008) for an introduction into counting), we can formally describe the setting as follows: There are $r_k = |\text{ran}(O_k)|$ different possible observations per partition $\mathcal{J}_k$, meaning, $\mathbf{o}_k$ can be encoded using a histogram:

$$\{(o_l, n_l)\}_{l=1}^{r_k}, o_l \in \text{ran}(O_k), n_l \in \mathbb{N}^0, \sum_l n_l = |\mathcal{J}_k|, \quad (5)$$

or $[n_1, \dots, n_{r_k}]$ for short. A histogram makes explicit a very basic insight: It does not matter which of the $|\mathcal{J}_k|$

agents observe a particular value o_l , only how many observe o_l . Basically, we have introduced a so-called counting random variable (CRV) for each partition with histograms as possible range values (Milch et al. 2008). Briefly coming back to the representation of each partition using a logical variable X_k , the idea is that for a random variable parameterised with a logical variable such as $O_k(X_k)$, we count how often a particular range value is assigned to any grounding of $O_k(X_k)$, leading to a CRV $\#_{X_k}[O_k(X_k)]$, with histograms of Eq. (5) as a range. Given this insight into lifting partition observations, the joint observation over a partitioned agent set turns into

$$\mathbf{o} = (h_1, \dots, h_K), h_k \in \text{ran}(\#_{X_k}[O_k(X_k)]). \quad (6)$$

The size of $\mathbf{o}$ reduces to $K \cdot \max_{k \in \{1, \dots, K\}} |\text{ran}(O_k)|$, which we assume to be much smaller than N . If we can expect the observations per partition to consist of the same observation for whole partitions, then storing only one value per group would be enough (histograms would be *peak-shaped* with one $n_l = |\mathcal{J}_k|$ and all other $n_{l'} = 0$), with the size of $\mathbf{o}$ reducing to K .

Actions can be treated analogously to observations: A joint action $\mathbf{a}$ is formalised as a sequence of partition actions A_k encoded as histograms using a CRV $\#_{X_k}[A_k(X_k)]$. The size of $\mathbf{a}$ reduces from N as well to $K \cdot \max_{k \in \{1, \dots, K\}} |\text{ran}(A_k)|$. With actions, we also might be able to go a step further: For indistinguishable agents, the same action leads to a maximum expected utility (unless further constraints take effect), yielding that in a policy, the same action applies for a partition, which means peak-shaped histograms, and requiring only one action to store per partition.

Since we assume that the agents within a partition are indistinguishable, the consequence for the transition model T and the sensor model Ω is that they both contain certain symmetric structures. There are a number of joint observation $(o_1, \dots, o_N)$ that are encoded with the same sequence of histograms (Eq. (6)), namely whenever, in each partition, the numbers n_l in a histogram occur but with a different permutation of agents yielding them. The same holds for joint actions. As such, the transition model and the sensor model will map to the same probability for those cases. Using a histogram allows for combining all the inputs mapping to the same probability into one input, reducing the size of T and Ω . Before we formalize what reduction we can get, we combine the above lifting of actions, observations, and models into the following definition of a lifted DecPOMDP:

Definition 5. *A lifted DecPOMDP is a tuple $(\{\mathcal{J}_k\}_k, S, \{\#_{X_k}[A_k(X_k)]\}_k, T, R, \{\#_{X_k}[O_k(X_k)]\}_k, \Omega)$, $k = 1$ to K , with*

- $\{\mathcal{J}_k\}_{k=1}^K$ a partitioning of an agent set $\mathbf{I}$,
- S a random variable with a set of states as range,
- $\#_{X_k}[A_k(X_k)]$ a CRV with a set of histograms as in Eq. (5) as range, with $\mathbf{A} = \times_{k=1}^K \text{ran}(\#_{X_k}[A_k(X_k)])$ the set of joint actions,

- $T(S', S, \mathbf{A}) = P(S' | S, \mathbf{A})$ a transition model,
- $R(S)$ a reward function,
- $\#_{X_k}[O_k(X_k)]$ a CRV with a set of histograms as in Eq. (5) as range, with $\mathbf{O} = \times_{k=1}^K \text{ran}(\#_{X_k}[O_k(X_k)])$ the set of joint observations, and
- $\Omega(\mathbf{O}, S) = P(\mathbf{O} | S)$ a sensor model.

Each partition $\mathcal{I}_k$ has a local policy π_k . The solution to a DecPOMDP is a joint policy $\boldsymbol{\pi} = (\pi_k)_{k=1}^K$.

Theorem 1. *The worst-case memory required for specifying a lifted DecPOMDP of Def. 5 is polynomial in N .*

Proof. The transition model $T(S', S, \mathbf{A})$ and sensor model $\Omega(\mathbf{O}, S)$ of a lifted DecPOMDP of Def. 5 have sizes S_T^{lif} and S_Ω^{lif} , respectively, that lie in

$$S_T^{lif} \in O(s \cdot s \cdot a_l^K) \quad \text{and} \quad S_\Omega^{lif} \in O(s \cdot o_l^K),$$

$s = |\text{ran}(S)|$, $a_l = \max_{k \in \{1, \dots, K\}} |\text{ran}(\#_{X_k}[A_k(X_k)])|$, and $o_l = \max_{k \in \{1, \dots, K\}} |\text{ran}(\#_{X_k}[O_k(X_k)])|$. The size of the histogram space for a CRV $\#_X[R(X)]$ with a range size r of R and a domain size of n of the logical variable X is given by $\binom{n+r-1}{r-1}$, which is bounded by n^r (Milch et al. 2008). Transferred to the lifted DecPOMDP setting, a_l and o_l can be described with n^a and n^o , respectively, with $n = \max_{k \in \{1, \dots, K\}} |\mathcal{J}_k|$, $a = \max_{k \in \{1, \dots, K\}} |\text{ran}(A_k)|$, and $o = \max_{k \in \{1, \dots, K\}} |\text{ran}(O_k)|$. Therefore, the worst case is given by

$$S_T^{lif} \in O(s \cdot s \cdot (n^a)^K) \text{ and } S_\Omega^{lif} \in O(s \cdot (n^o)^K), \quad (7)$$

which is no longer exponential compared to Eq. (3) in Lemma 1, but polynomial in $n < N$. $\square$

Assuming that $N \ll K$, then $S_T^{lif} \ll S_T^{dec}$ and $S_\Omega^{lif} \ll S_\Omega^{dec}$, even though $n^a > a$ and $n^o > o$. If requiring only peak-shaped histograms, we have the best case:

$$S_T^{lif} \in O(s \cdot s \cdot a^K) \quad \text{and} \quad S_\Omega^{lif} \in O(s \cdot o^K),$$

where the exponent of N is replaced by the exponent of K compared to Eq. (3), which with $N \ll K$ really showcases the reduction in memory required.

Having the representation no longer depends exponentially on the number of agents facilitates an opening towards tractable inference in lifted DecPOMDPs w.r.t. the size of the agent set: The lifted components may enable to also lift the calculations for a policy regarding these agents such that solving the inference problem of finding a policy no longer depends exponentially on the number of agents (tractability). Straightforwardly, one could use the value iteration approach for DecPOMDPs by building conditional plans for each partition and prune plans over all partitions.

Discussion

This section presents a nanoscale medical system, as described earlier, modeled as a lifted DecPOMDP and discusses the next steps for the formalism.

A Nanoscale System Application To model a nanoscale medical system as a lifted DecPOMDP, we need to specify the components of a lifted DecPOMDP. The following description models a nanoscale medical system as sketched in the brief introduction into such systems in the beginning of this paper.

The set of agents $\mathbf{I}$ with its K partitions consists of the different nanosensors and nanobots. There are κ types of nanosensors, each type reacting to one of κ different markers. So, for each type, there is a set of nanosensors, forming a partition $\mathcal{J}_k^\kappa$ in $\mathbf{I}$. For the nanobots, the setting is the same w.r.t. the types of messages that different types of nanobots react to. With ι message types, there are ι sets of nanobots, each forming a partition $\mathcal{J}_k^\iota$ in $\mathbf{I}$. Preliminary experiments have shown that each partition may have around 64,000 agents in such a nanoscale medical system, making the agent set at least of size $(\kappa + \iota) \cdot 64,000$.

Each type of agent basically has one action, which it can select to perform, and one possible observation. So, in terms of the model, there are two actions in each partition: (i) outputting its load, which are tiles for nanosensors and medication for nanobots, or (ii) doing nothing, and two observations: (i) for nanosensors, sensing a marker and for nanobots, receiving a message or (ii) sensing / receiving nothing.

The physical state space can be described in terms of the presence of markers and assembled messages of certain types. Considering only assembled messages is a simplification as messages undergo a series of different states themselves during assembly, but only the assembled message is of importance for a nanobot. With κ different markers and ι different messages, there are $2^\kappa \cdot 2^\iota$ states. Of course, other representations of the physical state space are possible, e.g., focusing on the medical context in a more detailed way.

With the given physical state space, the transition model T would need to model the presence of markers, which could possibly follow a Poisson distribution, as well as the presence of messages, which we assume to follow a log-normal distribution. To be able to compute a solution, further approximations might be necessary. The overall goal is that nanobots output their medication if corresponding markers are present, which the reward function needs to encode. The sensor model Ω would need to capture the probability of sensing correct inputs, which can vary greatly depending on outside influences. In general, we have the following sources of error: Nanosensors might sense a marker even though there is none or it might sense a marker of a wrong type as its own. Nanoagents might sense that they received a correct message even though the message is not there, the message is of another type, or the message is incorrectly assembled. Both types of nanodevices might also mistake a received input as there not being an input.

To close out this application example, let us consider the worst case space requirements of a nanoscale medical system modelled as a lifted DecPOMDP: If we consider four types of marker and one type of message, e.g.,

for detecting a specific disease, which means $\kappa = 4$ and $\iota = 1$, we have a state space of size $s = 2^4 \cdot 2^1 = 32$ whereas our agent set is of size $N = (4 + 1) \cdot 64,000 = 320,000$ partitioned into $K = 4 + 1 = 5$ partitions. With these parameters and in reference to Eqs. (3) and (7), the model sizes of T are

$$S_T^{dec} \in O(32 \cdot 32 \cdot 2^{320,000}) \quad (8)$$

$$S_T^{lif} \in O(32 \cdot 32 \cdot (64,000^2)^5) \quad (9)$$

and of Ω are

$$S_\Omega^{dec} \in O(32 \cdot 2^{320,000}) \quad (10)$$

$$S_\Omega^{lif} \in O(32 \cdot (64,000^2)^5) \quad (11)$$

in the worst case. Equations (8) to (11) highlight to what a large degree the number of agents represent the dominating parameter in a lifted DecPOMDP with a nanoscale system as an application.

Work in Progress The next big step after the inception of lifted DecPOMDPs is specifying and implementing an approach to solving lifted DecPOMDPs. A starting point lies in a value iteration approach for DecPOMDPs, which we lift for partitions of agents. The hypothesis is that the goal of *tractable inference w.r.t. agent numbers* is attainable given Theorem 1.

In a next step, we focus on the environment representation S . Assuming that there are a set of random variables $\{R_1, \dots, R_m\}$, with which we can describe the environment, the size of the state space is exponential in m , i.e., $O(r^m)$, with $r = \max_{i \in \{1, \dots, m\}} |ran(R_i)|$, which means for Eqs. (3) and (7) $s = r^m$. There are two aspects to pursue, *factorisation* and *lifting*. Factorisation refers to factorising a joint distribution into a set of local distributions exploiting (conditional) independences among the n random variables for a compact encoding, allowing for a complexity of $O(n \cdot r^w)$ where w refers to the so-called tree width, which basically denotes the largest number of arguments in an intermediate result during inference. Lifting could then apply to that set of local distributions, further compactifying the encoding, which enables tractable inference in terms of domain sizes under certain conditions (Taghipour 2013). For both aspects, the above mentioned (factorised) FO(PO)MDPs (Boutilier, Reiter, and Price 2001; Sanner and Boutilier 2007; Sanner and Kersting 2010) as well as the work on lifted online decision (Gehrke et al. 2019; Gehrke, Braun, and Möller 2019), which uses lifted factorised models, are a jumping-off point.

Turning to the application, an inconspicuous assumption comes into focus, namely, that the set of agents $\mathbf{I}$ is known. However, this assumption may not be true, possibly because the system designer cannot control how many agents arrive at a destination. This fact is especially true in a nanoscale system where not only the exact number of agents is not known but also how many of those agents function correctly. The usual practice of

using an excessive number of agents that practically guarantees a minimum threshold of agents does not work with medical systems where too much medicine can be harmful. Therefore, we need to keep partition sizes (and the overall number of agents) indefinite and possibly infer optimal sizes. From a modelling standpoint, since the agent set is a discrete, bounded set, we can use a beta-binomial distribution with hyperparameters α and β to model a probability distribution over possible set sizes for each partition.

The consequences for a given lifted DecPOMDP in terms of joint actions $\mathbf{A}$ and joint observations $\mathbf{O}$ lie in changed histograms. The more interesting consequence arise in the transition model T and sensor model Ω where the dimensions change. The models currently do not have further structure, which makes this problem challenging. Given a factorised representation with local distributions, inference in unknown universes (Braun and Möller 2019) is a starting point.

Conclusion

This paper presents lifted DecPOMDPs, lifting the agent set, allowing for a reduction of the worst-case dependency of the model from exponential to polynomial in the number of agents. Lifted DecPOMDPs work with a partitioning of the agent set where the agents of each partition are assumed to behave indistinguishably. Therefore, we can use well-established lifting formalisms such as counting to reduce the length of joint actions and joint observations as well as the size of the transition and sensor models. Lifted DecPOMDPs find their application in nanoscale systems, with the paper showcasing a medical diagnostics scenario.

Future work includes solving lifted DecPOMDPs, combining existing lifted solution approaches with the lifted representation, as well as recent advances in lifted online decision making. From the nanosystem side, an important aspect lies in a more detailed model of a nanonetwork. E.g., the stochastic behavior of the environment and the agents can be hard to model, making an analysis of network subclasses that can be compactly represented especially interesting.

References

- Ahmadi, B.; Kersting, K.; Mladenov, M.; and Nataraajan, S. 2013. Exploiting Symmetries for Scaling Loopy Belief Propagation and Relational Training. *Machine Learning* 92(1): 91–132.
- Akyildiz, I. F.; Brunetti, F.; and Blázquez, C. 2008. Nanonetworks: A new communication paradigm. *Computer Networks* 52(12): 2260–2279.
- Andersen, E. S.; Dong, M.; Nielsen, M. M.; Jahn, K.; Subramani, R.; Mamdouh, W.; Golas, M. M.; Sander, B.; Stark, H.; Oliveira, C. L.; et al. 2009. Self-assembly of a Nanoscale DNA Box with a Controllable Lid. *Nature* 459(7243): 73–76.
- Apsel, U.; and Brafman, R. I. 2011. Extended Lifted Inference with Joint Formulas. In *UAI-11 Proceedings*

of the 27th Conference on Uncertainty in Artificial Intelligence, 74–83. AUAI Press.

Boutilier, C.; Reiter, R.; and Price, B. 2001. Symbolic Dynamic Programming for First-order MDPs. In *IJCAI-01 Proceedings of the 17th International Joint Conference on Artificial Intelligence*, 690–697. IJCAI Organization.

Braun, T.; and Möller, R. 2018. Parameterised Queries and Lifted Query Answering. In *IJCAI-18 Proceedings of the 27th International Joint Conference on Artificial Intelligence*, 4980–4986. IJCAI Organization.

Braun, T.; and Möller, R. 2019. Exploring Unknown Universes in Probabilistic Relational Models. In *Proceedings of AI 2019: Advances in Artificial Intelligence*. Springer.

Gehrke, M.; Braun, T.; and Möller, R. 2019. Lifted Temporal Maximum Expected Utility. In *Proceedings of the 32nd Canadian Conference on Artificial Intelligence, Canadian AI 2019*. Springer.

Gehrke, M.; Braun, T.; Möller, R.; Waschkau, A.; Strumann, C.; and Steinhäuser, J. 2019. Lifted Maximum Expected Utility. In *Artificial Intelligence in Health*, 131–141. Springer.

Holtzen, S.; Millstein, T.; and Van den Broeck, G. 2019. Generating and Sampling Orbits for Lifted Probabilistic Inference. In *UAI-19 Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence*, 1–10. AUAI Press.

Lau, F.; Wendt, R.; and Fischer, S. 2021. DNA-Based Molecular Communication as a Paradigm for Multi-Parameter Detection of Diseases. In *4th ACM International Conference on Nanoscale Computing and Communication 2017 (ACM NanoCom’17)*. ACM.

Lau, F.-L. A.; BĀijther, F.; Geyer, R.; and Fischer, S. 2019. Computation of Decision Problems within Messages in DNA-tile-based Molecular Nanonetworks. *Nano Communication Networks* ISSN 1878-7789.

McCarthy, J. 1963. Situations, Actions, and Causal Laws. Technical report, Stanford University.

Milch, B.; Zettlemoyer, L. S.; Kersting, K.; Haimes, M.; and Kaelbling, L. P. 2008. Lifted Probabilistic Inference with Counting Formulas. In *AAAI-08 Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, 1062–1068. AAAI Press.

Nath, A.; and Domingos, P. 2009. A Language for Relational Decision Theory. In *Proceedings of the 6th International Workshop on Statistical Relational Learning*.

Niepert, M.; and Van den Broeck, G. 2014. Tractability through Exchangeability: A New Perspective on Efficient Probabilistic Inference. In *AAAI-14 Proceedings of the 28th AAAI Conference on Artificial Intelligence*, 2467–2475. AAAI Press.

Oliehoek, F. A.; and Amato, C. 2016. *A Concise Introduction to Decentralised POMDPs*. Springer.

Poole, D. 1997. The Independent Choice Logic for Modelling Multiple Agents under Uncertainty. *Journal of Artificial Intelligence* 94: 7–56.

Poole, D. 2003. First-order Probabilistic Inference. In *IJCAI-03 Proceedings of the 18th International Joint Conference on Artificial Intelligence*, 985–991. IJCAI Organization.

Rothmund, P. W. K. 2006. Folding DNA to Create Nanoscale Shapes and Patterns. *Nature* 440: 297–302.

Russell, S.; and Norvig, P. 2020. *Artificial Intelligence: A Modern Approach*. Pearson.

Sanner, S.; and Boutilier, C. 2007. Approximate Solution Techniques for Factored First-order MDPs. In *ICAPS-07 Proceedings of the 17th International Conference on Automated Planning and Scheduling*, 288–295. AAAI Press.

Sanner, S.; and Boutilier, C. 2009. Practical Solution Techniques for First-order MDPs. *Artificial Intelligence Journal* 173: 748–788.

Sanner, S.; and Kersting, K. 2010. Symbolic Dynamic Programming for First-order POMDPs. In *AAAI-10 Proceedings of the 24th AAAI Conference on Artificial Intelligence*, 1140–1146. AAAI Press.

Seeman, N. C. 1982. Nucleic Acid Junctions and Lattices. *Journal of Theoretical Biology* 99(2): 237 – 247. ISSN 0022-5193.

Srivastava, S.; Russell, S.; Ruan, P.; and Cheng, X. 2014. First-order Open-universe POMDPs. In *UAI-14 Proceedings of the 30th Conference on Uncertainty in Artificial Intelligence*, 742–751. AUAI Press.

Taghipour, N. 2013. *Lifted Probabilistic Inference by Variable Elimination*. Ph.D. thesis, KU Leuven.

Van den Broeck, G.; Taghipour, N.; Meert, W.; Davis, J.; and De Raedt, L. 2011. Lifted Probabilistic Inference by First-order Knowledge Compilation. In *IJCAI-11 Proceedings of the 22nd International Joint Conference on Artificial Intelligence*, 2178–2185. IJCAI Organization.

Winfrey, E.; Liu, F.; Wenzler, L. A.; and Seeman, N. C. 1998. Design and Self-assembly of Two-dimensional DNA Crystals. *Nature* 394(6693): 539.